

Utilizando *for*

O `for` repete um bloco **para cada item** de uma coleção: cada letra de um texto, cada item de uma lista, cada par de um dicionário. Nesta aula: percorrer coleções, `range()`, acumular resultados, `enumerate()` e `zip()`, `break` e `continue`, laços dentro de laços e percorrer dicionários.

`for item in coleção`

`range()`

Acumuladores

`enumerate` · `zip`

`break` · `continue`

`for` aninhado

`.items()`

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: for Statements, range(), break and continue; Looping Techniques), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Para cada item, *faça*

percorrer.py

```
frutas = ["maçã", "banana", "uva"]
for fruta in frutas:
    print("Eu gosto de", fruta)
print("Acabou o for")
```

SAÍDA

```
Eu gosto de maçã
Eu gosto de banana
Eu gosto de uva
Acabou o for
```

- ✓ A cada volta, a variável (`fruta`) recebe o próximo item
- ✓ O bloco com recuo roda uma vez por item
- ✓ Funciona com strings, listas, tuplas, sets e dicionários
- ✓ Nome no singular para o item, plural para a coleção: `for fruta in frutas`

Repetir *N vezes* com range()

range_ex.py

```
for i in range(5):
    print(i, end=" ")
print()
for i in range(1, 6):
    print(i, end=" ")
print()
for i in range(10, 0, -2):
    print(i, end=" ")
print()
print(list(range(3, 15, 4)))
```

SAÍDA

```
0 1 2 3 4
1 2 3 4 5
10 8 6 4 2
[3, 7, 11]
```

FORMA

GERA

range(5)

0, 1, 2, 3, 4

range(1, 6)

1 até 5 (o fim não entra)

range(10, 0, -2)

10, 8, 6, 4, 2

range(3, 15, 4)

3, 7, 11

Mesma regra das fatias (aula 07): o início entra, o fim não.
`list(range(...))` mostra os números gerados.

Somando *e procurando* o maior

acumular.py

```
notas = [8.5, 6.0, 9.5, 7.0]
soma = 0
for nota in notas:
    soma += nota
print("Soma:", soma, "| Média:", soma / len(notas))

maior = notas[0]
for nota in notas:
    if nota > maior:
        maior = nota
print("Maior:", maior)
```

SAÍDA

```
Soma: 31.0 | Média: 7.75
Maior: 9.5
```

- ✓ **Acumulador:** variável criada **antes** do for e atualizada dentro
- ✓ Soma começa em 0; produto começaria em 1; lista em `[]`
- ✓ Para o maior: começa com o primeiro e troca quando acha um maior

O Python já tem `sum()` e `max()` prontos, mas o padrão do acumulador serve para tudo que não tem função pronta.

enumerate() e zip()

enumerate_zip.py

```
nomes = ["Ana", "Bia", "Caio"]
notas = [9.0, 7.5, 8.0]
for i, nome in enumerate(nomes, start=1):
    print(f"{i}º {nome}")
for nome, nota in zip(nomes, notas):
    print(f"{nome}: {nota}")
```

SAÍDA

```
1º Ana
2º Bia
3º Caio
Ana: 9.0
Bia: 7.5
Caio: 8.0
```

- ✓ `enumerate()` entrega **posição e item**; `start=1` começa a contar do 1
- ✓ `zip()` anda em duas listas **juntas**, par a par
- ✓ Os dois devolvem tuplas, desempacotadas no for (aula 10)

Dica do guia (cap. 6): prefira `for item in lista` a `for i in range(len(lista))`. Se precisar da posição, `enumerate()`.

break *e* continue

break_continue.py

```
numeros = [4, 7, -1, 12, 0, 9]
for n in numeros:
    if n < 0:
        continue          # pula este e segue
    if n == 0:
        print("achou zero, parando")
        break             # sai do for
    print(n)
```

SAÍDA

```
4
7
12
achou zero, parando
```

- ✓ `continue` : pula o resto **desta** volta e vai para o próximo item (o -1 não apareceu)
- ✓ `break` : **sai** do for na hora (o 9 nunca foi visto)
- ✓ Úteis para ignorar dados ruins e parar quando achar o que procurava

for dentro de for

aninhado.py · MINI TABUADA

```
for linha in range(1, 4):  
    for coluna in range(1, 4):  
        print(linha * coluna, end="\t")  
    print()
```

1	2	3
2	4	6
3	6	9

Para cada **linha**, o for de dentro roda todas as **colunas**. $3 \times 3 = 9$ voltas no total.

dict_for.py · PERCORRER UM DICIONÁRIO COM IF

```
estoque = {"maçã": 12, "banana": 0, "uva": 5}  
for fruta, qtd in estoque.items():  
    if qtd == 0:  
        print(f"{fruta}: ESGOTADO")  
    else:  
        print(f"{fruta}: {qtd}")
```

maçã: 12
banana: ESGOTADO
uva: 5

Não adicione nem remova itens da coleção que está sendo percorrida: percorra uma cópia ou monte uma coleção nova.

A *tabuada*

Enunciado: peça um número e mostre a tabuada dele de 1 a 10, com os números alinhados.

Uma resposta possível

```
# Exercício: tabuada
n = int(input("Tabuada do: "))
for i in range(1, 11):
    print(f"{n} x {i:>2} = {n * i:>3}")
```

- ✓ `range(1, 11)` : o 11 não entra
- ✓ `{i:>2}` e `{n * i:>3}` alinham à direita (aula 05)
- ✓ Desafio: mostre as tabuadas do 1 ao 5, com dois for

SAÍDA (DIGITOU 7)

```
Tabuada do: 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```

Teste o que *ficou*

1. Quais números `range(2, 6)` gera?

A) 2, 3, 4, 5, 6

B) 2, 3, 4, 5

C) 3, 4, 5, 6

D) 2, 6

B. O fim não entra.

2. O que o `continue` faz?

A) Sai do `for`

B) Pula para o próximo item

C) Repete o item

D) Para o programa

B. `break` é que sai.

3. Para ter posição e item juntos:

A) `zip()`

B) `enumerate()`

C) `range()`

D) `items()`

B. `enumerate(lista, start=1)`.

4. Onde criar a variável que acumula a soma?

A) Dentro do `for`

B) Antes do `for`

C) Depois do `for`

D) Tanto faz

B. Dentro, ela zeraria a cada volta.

Um item *de cada vez*

O `for` percorre qualquer coleção; `range()` gera sequências de números; acumuladores somam e procuram; `enumerate()` e `zip()` dão posição e pares; `break` sai e `continue` pula; laços aninhados percorrem tabelas.

`for x in coleção`

`range(início, fim, passo)`

Acumulador

`enumerate · zip`

`break · continue`

Próxima aula → 15 · Utilizando While

Fontes: docs.python.org/3/tutorial/controlflow.html (`for`, `range`, `break`, `continue`); docs.python.org/3/tutorial/datastructures.html#looping-techniques, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.